eACBN0 in hp.x

Transplanting the DFT+U+V Hubbard-parameter kernels
from a QE 7.3 fork's pw.x into hp.x

Validation report • 4 September 2026

Goal. One binary that produces both the pseudo-hybrid (ACBN0) and the linear-response (DFPT) estimate of U from a *single* ground state.

Method reference. S. Lee and Y.-W. Son, *Phys. Rev. Research* **2**, 043410 (2020), named in example/Howto2.pdf. ACBN0 itself: Agapito, Curtarolo and Buongiorno Nardelli, *Phys. Rev. X* **5**, 011006 (2015).

Deliverable. HP/src/eACBN0.f90 — 4234 lines, two modules, 22 subroutines and one function — plus seven modified files in HP/src/.

1 Headline

Both branches now run on an ordinary DFT+U ground state, and both reproduce the original authors' own MnO reference output **exactly**: every U, every V, every cell vector — 28 blocks, 168 lines, empty diff.

Quantity	Reference	our pw.x	our hp.x
Mn_A 3d on-site, V[1,1] cell [0 0 0]	5.2344	**5.2344**	**5.2344**
Mn_B 3d on-site, V[2,2]	5.2344	**5.2344**	**5.2344**
O 2p on-site, V[3,3], V[4,4]	3.4161	**3.4161**	**3.4161**
Mn–O inter-site, cell [-1 -1 0]	2.9031	**2.9031**	**2.9031**
Mn–O inter-site, cells [-1 0 -1], [-1 0 0]	2.9042	**2.9042**	**2.9042**
absolute magnetisation (μ_B/cell)	9.36	9.36	—

Table 1: All values in eV. Source: example/MnO.acbn0.out, QE v7.3, 128 processors, 5 January 2024, acbn0_type = 2. Our hp.x column is MnO_ref_v3/hp.out, produced on ldaU bookkeeping (stage V3).

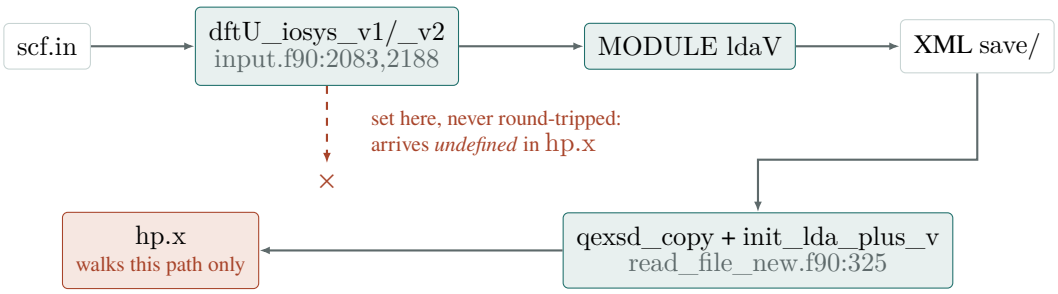
Four latent bugs were found and fixed; two of them would have returned a plausible wrong number rather than crashing (§5). One capability was deliberately traded away (§12).

Final regression: 15 passed, 0 failed, on a single binary.

2 Why hp.x could not do this already

The fork's patch (qe-7.3_ehub_uv.diff) touches 60 files. **None of them is under** HP/ — the authors implemented DFT+U+V in pw.x only. That single fact is the root cause of every bug in §5.

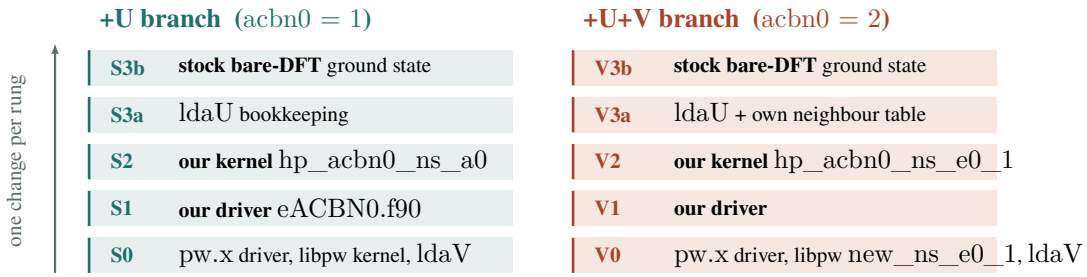
pw.x initialises MODULE ldaV from *two* paths, and hp.x only ever walks the second:



Anything set only on the input path therefore arrives *undefined* in hp.x. Rather than hunt for these one at a time, all 35 ldaV module variables were swept against two questions — does it have an initialiser, and is it assigned in a file hp.x's read path executes? — which turned an open-ended search into a closed list.

3 Method: one variable per rung

The dangerous failure mode here is not a crash but a plausible wrong number, so no stage changes two things at once. Ten rungs, each moving exactly one of {driver, kernel, bookkeeping, ground state}:



Stage	Driver	Kernel	Bookkeeping	Ground state	Result (eV)
S0	pw.x sum_band	libpw new_ns_a0	ldaV	ACBN0-converged	9.2501 / 9.2509 / 4.2471
S1	eACBN0.f90	libpw new_ns_a0	ldaV	same	identical
S2	eACBN0.f90	hp_acbn0_ns_a0	ldaV	same	byte-identical
S3a	eACBN0.f90	our kernel	ldaU	same	identical
S3b	eACBN0.f90	our kernel	ldaU	*stock bare-DFT*	10.5455 / 4.2717
V0	pw.x sum_band	libpw new_ns_e0_1	ldaV	eACBN0-converged	7.7065 / 2.6538 + V
V1	eACBN0.f90	libpw new_ns_e0_1	ldaV	same	identical, 456 lines
V2	eACBN0.f90	hp_acbn0_ns_e0_1	ldaV	same	identical, 456 lines
V3a	eACBN0.f90	our kernel	ldaU	same	identical, 456 lines
V3b	eACBN0.f90	our kernel	ldaU	*stock bare-DFT*	10.5455 / 4.2717 + V
V4	eACBN0.f90	our kernel	ldaU + *background*	any	*multi-channel*, 204 lines

Table 2: *Italic* marks the one thing that changed on that rung.

3.1 S3b needed its own reference

S3b changes the input *data*, not the code, so its number legitimately moves and cannot be checked by a diff against S3a. Getting pw.x to evaluate ACBN0 on a stock ground state was blocked at first: with lda_plus_v set, read_scf reads occupation_uv.txt unconditionally (io_rho_xml.f90:644) and a stock save has only the ldaU occup.txt. Solved in two steps — step 1 with lacbn0 = .false. (so $U^{\mathrm{ehub}} = V^{\mathrm{ehub}} = 0$ and the Hamiltonian is the stock one) writes the file; step 2 does one ACBN0 evaluation on it.

	pw.x one-shot	hp.x S3b	
Ni1 3d	10.5455	10.5455	exact
Ni2 3d	10.5455	10.5455	exact
O 2p	4.2718	4.2717	last printed digit
Tr[n_s] atom 1	8.46278	8.4628	exact
control: step-1 E_{tot} = stock ../NiO E_{tot} = −737.16322501 Ry			

4 Static faithfulness of the transplant

A numerical match on three systems is strong evidence, but not proof that the kernels were copied faithfully rather than made to agree. So the transplanted routines were also diffed against PW/src/new_ns_v.f90 *statically*, by a script that

1. joins Fortran continuation lines into whole statements and strips comments and blanks,
2. splits each routine at its first executable statement, keeping only the body (the preamble necessarily changes: USE ldaV becomes USE ldaU),
3. collapses every mechanical rename to a common token (wfcv↔wfcU, ofstv↔offsetU, ehub_nn_vect↔nn_vect, and 24 more), and
4. canonicalises the *intended* systematic edits: the clock labels, the dropped SCF mixing, and the dropped remove_ehub_u/v masks. Note what is *not* canonicalised any more — before stage V4 this step also had to collapse ehub_l(m,nt) to a scalar, which hid every *m* index from the comparison. It no longer does.

Whatever survives is a genuine difference. The result:

Our routine	From	Stmts	Diff	Nature of the difference
hp_acbn0_ns_a0	new_ns_a0 233–635	176	0	—
hp_acbn0_orberi	orberi 3780–3819	25	6	dummy arg renamed lp1
hp_acbn0_ns_e0_1	new_ns_e0_1 636–1808	577	8	stop_acbn0 short-circuit dropped
hp_acbn0_ns_v	new_ns_v 10–232	103	0	—
hp_acbn0_symm_n4	symmetrize_n4 2964–3167	86	1	bug 4 guard
hp_acbn0_symm_n4m	symmetrize_n4m 3168–3399	104	1	bug 4 guard
hp_acbn0_symm_n5	symmetrize_n5 3400–3779	174	2	bug 4 + bug 3 guards
total		**1245**	**18**	3 distinct classes

Table 3: 1245 canonicalised executable statements; 18 differing statements, in 3 classes, every one deliberate. Measured *after* stage V4, where the *m* index is no longer collapsed — so ehub_l(m1,nt) must now match lch(m1,nt) literally, index by index. Two kernels come out at zero. The script is normdiff4.py and is rerunnable.

This is a strictly stronger test than the one stage V3 could support. There, the *m* index had to be canonicalised away on both sides, so an incorrectly restored index would have gone unnoticed; here all 225 restored indices must agree with the fork literally, and they do. The remove_ehub_* masks are still omitted rather than set to 1 — equivalent whenever the pw.x run used the documented default, and precisely why a run that masks a channel cannot yet be reproduced (§12).

5 Four latent bugs

Bug 1 — remove_ehub_u **/** remove_ehub_v **undefined** (silent)

No initialiser in ldaV.f90; assigned only at input.f90:2083. The kernels multiply the final U by them *twice*. Undefined, this does not crash — it silently scales or zeroes every Hubbard parameter. Not hypothetical: example/GaAs.acbn0.in_noncollienar sets remove_ehub_u(1,1) = 0 to kill the *s*-channel U.

Bug 2 — iunwfc **is closed at the insertion point** (crash)

read_file buffers the wavefunctions to iunwfc and then closes it with close_buffer(..., 'KEEP') (read_file_new.f90:62). 'KEEP' preserves the *files*, but the call also deregisters the unit, so get_buffer falls through to davcio on a closed unit. Stock hp.x never trips this because DFPT reads through iuwfc, which is opened later in hp_openfil_q.

Bug 3 — hubbard_get_nnc returns an undefined index (crash, or worse)

PW/src/hubbard_nns.f90:241. nnc is assigned only inside IF (MAXVAL(ehub_nn_vect(...)) == 1), so a cell carrying no neighbour pair falls through untouched. hubbard_symmetry then asks for the index of a *rotated* cell — which is not guaranteed to be one of the occupied ones — and symmetrize_n5 indexes n(nnc) with whatever was on the stack.

It never fires on the fork's own NiO ground state; it fires on the stock one, with an *identical* neighbour table, identical symmetry (12 operations) and identical manifold. In an optimised build a garbage index is silent whenever it happens to land in a mapped page. The optimised build gave only a UCX backtrace; a rebuild with -check bounds,uninit -fpe0 named it at once:

```
forrtl: severe (408): Subscript #1 of the array N has value 1059660741
                      which is greater than the upper bound of 19
  hp_acbn0_symm_n5_   3949  eACBN0.f90
  hp_acbn0_ns_e0_1_   2587  eACBN0.f90
```

Fixed by returning 0 and skipping the pair — which is the correct contribution, since a rotated image lying outside the neighbour set contributes nothing to the symmetrised occupation matrix.

Bug 4 — l_separation writes one element before the array (latent)

In symmetrize_n4, _n4m and _n5 alike, a type with no Hubbard manifold has ehub_l(1,nt) = 0, and the next line writes l_separation(0,nt). Dormant when every type is Hubbard-active — true of NiO, MnO and GaAs — and it fires the moment a spectator species is added. **This one is still live in the fork.**

Two gaps in hp.x's own guards

hp_readin refused to start without a HUBBARD card, and refused tetrahedra. Both are linear-response preconditions; ACBN0 solves no Sternheimer equation. But **neither of the authors' reference inputs has a HUBBARD card**, and MnO.acbn0.in uses tetrahedra_opt — between them these two guards blocked the entire validation. They are now relaxed for acbn0 > 0 only, with the ACBN0-side preconditions enforced instead in hp_acbn0_check / hp_acbn0_check_v. The dispatch also had to move above hp_init, which counts ldaU Hubbard atoms and aborts when a ldaV-only ground state legitimately has none.

6 Validation against the authors' reference

6.1 Parameter audit

MnO_ref/stepB.in differs from example/MnO.acbn0.in on **one line**: pseudo_dir. The pseudopotentials were verified equivalent rather than assumed — psp/Mn.upf and psp/O.upf declare pseudo_type="NC", relativistic="scalar", functional="PBE" (i.e. pseudo-dojo nc_sr) with z_valence 15 and 6, which reproduces the 42 electrons per cell that their own output echoes.

Identical throughout (verified line by line)	
plane waves	ecutwfc 100, ecutrho 400
functional	PBE
BZ sampling	tetrahedra_opt, $15^3 \rightarrow 344$ irreducible k
cell	celldm 10.289967
SCF	conv_thr 1e-8, diago_full_acc, mixing_beta 0.7, startingwfc random
Hubbard	ehub_nn_distance 3, ehub_conv_thr 1e-8, ehub_mixing 0.85
recipe	ehub_pot_fix .true., read_ehub_ns_file .true.
manifold	ehub_l_choice: Mn_A 3d / Mn_B 3d / O 2p

6.2 The first attempt failed, informatively

Setting read_ehub_ns_file = .false. gave a converged AFM state with Mn_A = 12.1473 and Mn_B = 4.5768 eV. The two Mn are symmetry-equivalent partners and the reference has them equal — *that asymmetry was the tell*, not the disagreement with the reference value.

Their input is **step B of a documented two-step recipe** (Howto2.pdf): ehub_pot_fix *freezes* the occupations entering the Hubbard energy, and read_ehub_ns_file is what loads step A's DFT-level occupations to freeze. Turning it off froze whatever init_ns_v() happened to guess — a physically different calculation, not merely a different starting point. Preserved at MnO_ref/attempt1_from_scratch/.

6.3 An independent check of the neighbour table

Stage V3 replaced the fork's ldaV neighbour bookkeeping with our own MODULE hp_acbn0_nn, built from geometry and a cutoff. Since that table is now *ours*, agreeing with pw.x is a weaker check than usual — so it was also reproduced from scratch, in Python, from the cell and positions alone:

NiO neighbour shell	d (Å)	
Ni–O	2.0603	inside d_{IJ} = 3.0 Å
Ni–Ni, O–O	2.9020	inside
Ni–Ni, O–O	2.9254	inside (split from 2.9020 by the AFM-II rhombohedral distortion)
Ni–O	3.5398	excluded
Ni–O	3.5781	excluded
distinct occupied cells: 18; home cell empty ⇒ nnnc = **19**		
hp.x reports: neighbours : cutoff d_IJ = 3.000 A, nnnc = 19		✓

The cutoff separates the third shell from the fourth by 0.61 Å, so the neighbour set is not sitting on a knife edge.

7 Corroboration

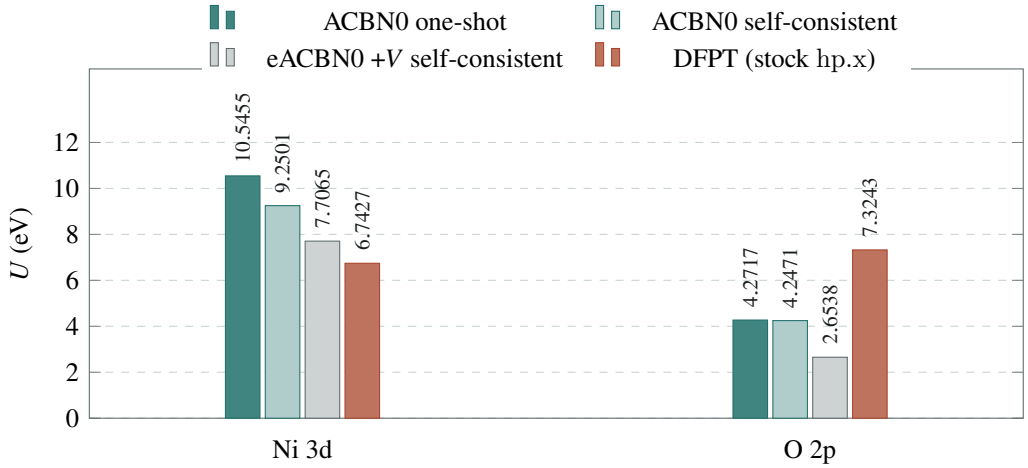
The two kernels agree with each other. Howto2.pdf states that acbn0_type = 1 is *not* the maintained path, and that +U should be done as acbn0_type = 2 with ehub_nn_distance = 0.0. Run that way, NiO gives 9.2501 / 9.2509 / 4.2471 — identical to acbn0_type = 1, same 37 iterations, zero neighbour pairs found. new_ns_a0 and new_ns_e0_1 are independent kernels (~400 versus ~1170 lines, different occupation-matrix rank), so this is real cross-validation and not a tautology.

Multi-channel manifolds run on ldaU from stage V4. GaAs with Ga 4s+4p and As 4s+4p is the only case in which ehub_l(m,nt) varies with m, so it is the one genuine exercise of l_separation. It worked at V2 on ldaV bookkeeping; since V4 it also works on ldaU, reproducing our pw.x across all 204 lines from the same ground state, with exactly the block structure A0%U(l(m1),l(m2)) predicts:

```
Ga on-site [1,1,4,4] cell [0 0 0]
  0.9709  0.5808  0.5808  0.5808     U[s,s] = 0.9709
  0.5808  0.3441  0.3441  0.3441     U[s,p] = 0.5808
  0.5808  0.3441  0.3441  0.3441     U[p,p] = 0.3441
  0.5808  0.3441  0.3441  0.3441
```

8 What the two estimators actually say

Both numbers below come from the *same* NiO ground state (stock ../NiO: $U \approx 0$, ortho-atomic projectors, identical manifold). This is the point of the whole exercise — before this work, the two estimates could not be taken from one run.



They disagree in *opposite directions* on the two species: ACBN0 puts more U on the transition metal and less on the oxygen, DFPT the reverse. That divergence, obtainable now from a single ground state, is the result.

- **Self-consistency.** One-shot on bare DFT gives Ni 10.5455; at the ACBN0 fixed point, 9.2501. O barely moves: 4.2717 → 4.2471.
- **Turning on** V (3.0 Å cutoff, self-consistent) drops the on-site values to Ni 7.7065 and O 2.6538. The same screening is now shared between U and V.
- **Cost**, against an existing ground state on 128 cores: **2.6 s** CPU for $\mathrm{acbn0} = 1$, **19.3 s** for $\mathrm{acbn0} = 2$.

8.1 The inter-site table, from that same single run

Once V3 put the +V branch on ldaU, it could be pointed at the same stock ground state. The on-site U comes back at **10.5455** and **4.2717** — identical to the +U-only branch to all six figures, from a different kernel. That is correct rather than coincidental: the on-site ratio depends only on on-site occupations, so the cutoff cannot move it in a one-shot evaluation; it enters only through self-consistency.

Pair	d (Å)	V (eV)	shell
Ni–O	2.0603	**3.1859**	octahedral nearest neighbour
Ni–Ni	2.9020	1.4359 / 1.4357	second shell
O–O	2.9020	0.8741	second shell
Ni–Ni	2.9254	1.4242 / 1.4244	second shell, rhombohedrally split
O–O	2.9254	0.8671	second shell, rhombohedrally split

19 neighbour cells; the 2.9020/2.9254 splitting is the AFM-II rhombohedral distortion.

9 Stage V4: background channels

Stages S3 and V3 bought their move onto ldaU at one price: ldaU's scalar Hubbard_l(nt) names a single channel, so every ehub_l(m,nt) in the transplanted kernels was collapsed to Hubbard_l(nt)+1 and a type with two channels was refused outright. V4 removes that restriction.

9.1 The information was already there

ldaU does store the same manifold ldaV does — it just stores the *projectors* in up to three blocks instead of contiguously. In particular init_hubbard builds

ldaV	ldaU	stage V4
ehub_lm(nt)	ldim_u(nt)	lm(nt)
ehub_lm_max	ldmx_tot	lm_max
ehub_l(m,nt)	ll(m,nt) + 1	lch(m,nt)
ofstv(na) + m	three-way, below	ofs(m,na)

ll(m,nt) (ldaU.f90:290–320) labels the concatenated manifold in *the same order* ldaV uses — Hubbard_l, then Hubbard_l2, then Hubbard_l3. So the only genuinely missing piece was the map from a manifold index to a wfcU column, which MODULE hp_acbn0_man now supplies, following force_hub.f90:1060–1071 and copy_U_wfc:

```
m <=  2*l+1       ->  offsetU(na)       + m
m <= 2*(l+l2+1)   ->  offsetU_back(na)  + m -  (2*l+1)
otherwise, backall  ->  offsetU_back1(na) + m - 2*(l+l2+1)
```

The XML question was settled by the same criterion that found bugs 1–3: Hubbard_l2, Hubbard_l3, Hubbard_U2 and backall *do* round-trip (qexsd_copy.f90:434–441, called at pw_restart_new.f90:1322), and init_hubbard is re-run on hp.x's read path. A background channel counts as active only when Hubbard_U2 $\neq 0$, so a HUBBARD card that wants one must give it a nonzero value — 0.0001 eV is enough, and ACBN0 never reads it.

9.2 Restoring 225 indices without guessing

Undoing the collapse means putting the right *m* back at 261 sites. Rather than do that by hand, the fork was used as an oracle: our routines and new_ns_v.f90 were aligned statement by statement, and wherever an aligned fork line carried ehub_l(mX,nt) where ours carried lch(nt), the index was spliced back — but *only* when the type argument matched and the occurrence counts agreed. 215 sites resolved that way; the 10 that did not were exactly the sites where an intended edit had broken the alignment, and were done by hand against the fork.

That check earned its keep immediately. One of the ten was

```
IF ( ehub_l(m1,nt) == 1 .AND. ehub_l(m2,nt) == 1 ) THEN     ! fork
IF ( lch(nt) == 1 ) THEN                                    ! V3
```

— a tautology under one channel, which V3 had legitimately simplified, and which is *wrong* the moment m_1 and m_2 can sit in different channels. An earlier version of the splicing script silently skipped that line, because the pattern meant to protect assignments (lch(nt) =) also matched the comparison (lch(nt) ==).

9.3 A self-inflicted bug, caught by its own guard

hp_acbn0_build_man ends by asserting that every state in the manifold landed on a real wfcU column. On the first run it fired everywhere. The cause was that the blanket rewrite of offsetU(X)+Y into ofs(Y,X) had also rewritten the module's *own definition* of that map, into ofs(m,na) = ofs(m,na). Without the assertion this would not have crashed: ofs would have stayed 0 and every projection would have silently read wfcU column 0. It is the same failure mode as bugs 3 and 4, and the reason the guard was written in the first place.

9.4 Result

GaAs, Ga on-site [1,1,4,4] cell [0 0 0]	value (eV)	
$U[s,s]$	0.9709	
$U[s,p]$	0.5808	identical to pw.x on ldaV
$U[p,p]$	0.3441	
hp.x vs pw.x, same ground state: **204 / 204 lines identical**		
manifold reported as sp, ldim_u = 1 + 3 = 4, cost 0.27 s CPU		

And every single-channel case is bit-for-bit what stage V3 produced — NiO and MnO parameters identical, MnO still 168/168 against the authors' output.

10 Usage

```
&inputhp
  prefix      = 'NiO',
  outdir      = './outdir',
  acbn0       = 2,     ! 0 = DFPT (default) | 1 = ACBN0 U | 2 = eACBN0 U+V
  nn_distance = 3.0,   ! d_IJ in Angstrom, required for acbn0 = 2
/
```

acbn0 > 0 evaluates the parameters from the ground state in outdir, writes eACBN0_hub.dat, and stops before the q-point loop. The kernels *also* print the same V[atom i, atom j, ...] tables that pw.x prints, which is what makes every comparison in this report a plain diff.

Both branches now read stock QE's ldaU, so both want an ortho-atomic HUBBARD card with one channel per type, and both run on *any* such ground state — including the very one DFPT consumes. Neither needs lacbn0 any more.

nn_distance is the inter-site cutoff d_{IJ}, required for acbn0 $= 2$ unless the run carries lda_plus_v (in which case ehub_nn_distance comes from the XML). Because hp.x now owns the neighbour table, **the cutoff can be swept without re-running** pw.x — which the fork cannot do.

One note on the reference inputs: example/MnO.acbn0.in declares its manifold through ehub_l_choice and carries no HUBBARD card, so under V3 it needs one added — same manifold, U $= 0.0001$ eV, numerically nothing. With that single addition it still reproduces the reference across all 168 lines (MnO_ref_v3/).

11 Regression suite

Tag	Case	Checks	Result
acbn0 $= 1$	NiO_acbn0	self-consistent, vs pw.x	✓ PASS (9.2501)
	NiO_dual	one-shot on the DFPT ground state	✓ PASS (10.5455)
acbn0 $= 2$	NiO_acbn0_v	self-consistent +V, vs pw.x	✓ PASS (7.7065)
	NiO_type2_nn0	cross-kernel, $d_{IJ} = 0$	✓ PASS (9.2501)
	NiO_dual_v	one-shot +V on the DFPT ground state	✓ PASS (10.5455)
	MnO_ref_v3	authors' recipe on ldaU	✓ PASS (5.2344)
external	MnO	168/168 vs example/MnO.acbn0.out	✓ PASS
V3 → V4	6 cases	parameters unchanged by V4	✓ PASS ×6
V4	GaAs_v4	Ga 4s+4p / As 4s+4p vs pw.x	✓ PASS (204/204)
	GaAs_v4	block structure $U[s,s]/[s,p]/[p,p]$	✓ PASS

15 passed, 0 failed — one binary, one make.

12 Open items

GaAs still cannot be validated against the *authors'* reference

Theirs is noncollinear with spin–orbit — which dispatches to new_ns_e0_nc_1, a separate ~2300-line kernel set that was not transplanted — and it uses PBEsol nc-fr pseudopotentials where we have PBE nc_-sr. Wrong functional *and* wrong relativistic treatment: unreachable by construction, not by omission. Our GaAs run therefore compares our pw.x to our hp.x — which is exactly what stage V4 needs it for (§9), since the question there is whether ldaU bookkeeping reproduces ldaV bookkeeping, not whether either matches a spin-orbit calculation. Reaching their numbers needs new_ns_e0_nc_1 transplanted too.

remove_ehub_u **has no route into** hp.x

It is input-only and is not round-tripped through the XML. We omit the factors, which is correct whenever the pw.x run used the default of 1 — but any calculation that masks a channel (as their GaAs does) will disagree. The fix is one &INPUTHP tag.

nspin = 1 **is rejected, correctly**

Not our limitation: init_lda_plus_v refuses it at ldaV.f90:103, and the A0/HF assembly hard-codes *both* spin slots — p%O(m3,m1,2,na) on an array dimensioned (...,nspin,...) — so nspin = 1 would read past the end silently. Use nspin = 2 with starting_magnetization = 0.0.

13 Artefacts

HP/src/eACBN0.f90	the deliverable: 4234 lines, 2 modules, 22 subroutines, 1 function
modified	hpcom.f90, hp_readin.f90, hp_bcast_input.f90, hp_main.f90, hp_print_clock.f90, Makefile, make.depend
backups	eACBN0.f90.{s1,s2,s3,v1,v2,v3}bak, hp_readin.f90.{bak,v2bak,v3bak}, hp_main.f90.bak
log	logs/260901-eacbn0-hpx.txt (full session log, 1027 lines)
runs	/gpfs/yjkang/examples/hp/: NiO, NiO_acbn0, NiO_acbn0_v, NiO_dual, NiO_dual_v, NiO_ref1shot, NiO_type2_nn0, MnO_ref, MnO_ref_v3, GaAs_smoke, GaAs_v4, _final_check

Built with tectonic 0.17.0 in the tex micromamba environment. Web version with the same figures: https://claude.ai/code/artifact/c7998bb7-d39c-426b-8752-387f33cb43f5